AD23B31 - IMAGE PROCESSING AND COMPUTER VISION

HELMET AND SAFETY GEAR DETECTION FOR TWO-WHEELER RIDERS

A MINI PROJECT REPORT

Submitted by

DHARSHINI R S

2116230701076

March, 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**HELMET AND SAFETY GEAR DETECTION FOR TWO-WHEELER RIDERS**" is the bonafide work of **2116230701076** who carried out the work under my supervision.

Certified further that to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

The project work has been carried out under my supervision and to my satisfaction. This work is original and has not been submitted elsewhere for the award of any degree or diploma.

SIGNATURE
Dr.S.Madhusudhanan
Associate Professor
Department of Computer Science Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voice Examination held on _____

Internal Examiner External Examiner

ABSTRACT

Road traffic accidents involving two-wheeler riders remain a critical public safety challenge, especially in countries like India where motorcycles and scooters are the primary mode of transportation. Non-use of helmets is a major contributing factor to fatalities in such accidents. This project presents SENTINEL, an automated helmet and safety gear detection system designed to assist traffic enforcement authorities with real-time compliance monitoring.

The system employs a hybrid two-stage detection pipeline. In the first stage, YOLOv8n is used to detect persons in the input frame. In the second stage, a tight head region of interest (ROI) is extracted and classified using a Support Vector Machine (SVM) trained on Histogram of Oriented Gradients (HOG) and Local Binary Pattern (LBP) features. The model is trained on two combined annotated datasets — the BikesHelmets dataset (Pascal VOC XML format) and an Indian road dataset (YOLO txt format) — achieving an accuracy of 94.27% and an AUC-ROC of 0.9832 on the combined test set.

The system is deployed as a Flask-based web application named SENTINEL, supporting image upload, video processing, and live webcam surveillance. The solution is computationally efficient and operable on standard CPU-based hardware, making it suitable for practical deployment in traffic monitoring scenarios.

Keywords: Helmet Detection, HOG, LBP, SVM, YOLOv8, Flask, Safety Gear Detection, Two-Wheeler, Computer Vision, Road Safety.

TABLE OF CONTENTS

S.No	Contents	Page No.
1.	Introduction	
2.	Problem Statement	
3.	Objectives	
4.	Literature Survey	
5.	System Requirements	
6.	Methodology	
7.	System Architecture	
8.	System Flow Diagram	
9.	Technology Stack	
10.	Working of the System	
11.	Implementation Details	
12.	Results and Discussion	
13.	Advantages	
14.	Limitations	
15.	Future Enhancements	
16.	Conclusion	
17.	References	

CHAPTER 1: INTRODUCTION

1.1 Background

Two-wheeler road accidents are a leading cause of fatalities globally, particularly in developing nations where motorcycles constitute the dominant mode of commute. Among the primary factors amplifying injury severity in such accidents is the failure of riders to wear helmets. Despite legislative mandates in several countries, enforcement remains inconsistent due to a lack of automated monitoring infrastructure.

Traditional methods of helmet compliance monitoring rely heavily on manual inspection by traffic personnel, which is both inefficient and prone to human error. There exists a critical need for an intelligent, automated system capable of detecting helmet usage in real time from camera feeds, enabling scalable and consistent traffic enforcement.

1.2 Motivation

Advances in computer vision and machine learning have made it increasingly feasible to build automated surveillance systems that can operate on standard hardware. Classical feature extraction methods such as Histogram of Oriented Gradients (HOG) and Local Binary Patterns (LBP), combined with discriminative classifiers like Support Vector Machines (SVM), offer a computationally efficient approach to image classification without the need for deep learning infrastructure.

Simultaneously, state-of-the-art object detectors such as YOLOv8 enable fast and accurate person localization in real-world scenes. The fusion of these two paradigms motivates the development of SENTINEL — a practical, deployable helmet detection system that leverages the strengths of both classical and modern computer vision approaches.

1.3 Project Overview

SENTINEL is an automated helmet and safety gear detection system developed as a Flask-based web application. The system follows a two-stage pipeline: YOLOv8n first detects persons in the input frame, after which a head region of interest (ROI) is extracted and classified by an SVM model trained on HOG and LBP features.

The system supports three input modes — image upload, video file analysis, and live webcam surveillance — and outputs annotated results indicating whether each detected rider is wearing a helmet or not, along with confidence scores and summary statistics.

1.4 Scope of the Project

The scope of this project includes:

- Two-stage person detection and helmet classification pipeline
- Feature extraction using HOG and LBP descriptors

- SVM-based binary classification with probability calibration
- Training on two annotated helmet datasets with data augmentation
- Flask web application supporting image, video, and webcam inputs
- Detection result visualization with bounding boxes and confidence scores

CHAPTER 2: PROBLEM STATEMENT

Non-helmet usage among two-wheeler riders is a persistent road safety issue with severe consequences. Manual enforcement is insufficient given the volume of traffic in urban environments. Existing automated systems are often computationally expensive, requiring GPU infrastructure that is impractical for real-world deployment at traffic checkpoints.

There is a pressing need for an intelligent and affordable helmet detection system that:

- Detects two-wheeler riders and classifies helmet compliance in real time.
- Operates efficiently on standard CPU-based hardware without GPU dependency.
- Generalizes well across diverse lighting conditions, rider poses, and helmet types.
- Is deployable through an accessible web-based interface for practical use by traffic authorities.

This project addresses these challenges by developing SENTINEL, a two-stage detection system combining YOLOv8 person detection with HOG + LBP + SVM helmet classification, deployed as a Flask web application.

CHAPTER 3: OBJECTIVES

3.1 Primary Objectives

- To detect persons on two-wheelers using the YOLOv8n object detection model.
- To extract the head region of interest (ROI) from each detected person bounding box.
- To classify helmet compliance using an SVM trained on HOG and LBP feature descriptors.
- To train and evaluate the model on two combined, annotated datasets achieving high accuracy.
- To deploy the system as a Flask web application supporting image, video, and webcam inputs.

3.2 Secondary Objectives

- To implement data augmentation strategies to improve model robustness.
- To apply PCA-based dimensionality reduction to optimize classification efficiency.
- To calibrate SVM probability outputs and determine an optimal classification threshold.
- To visualize detection results with annotated bounding boxes and confidence scores.
- To design the system in a modular manner for future extensibility and enhancement.

CHAPTER 4: LITERATURE SURVEY

4.1 Feature-Based Object Detection

Dalal and Triggs (2005) introduced the Histogram of Oriented Gradients (HOG) descriptor, which captures edge and gradient structure information to represent object appearance. HOG demonstrated excellent performance for pedestrian detection and has since become a foundational feature for classification tasks. Its robustness to moderate illumination changes makes it well suited for outdoor surveillance applications such as helmet detection.

Ojala et al. (2002) proposed Local Binary Patterns (LBP), a texture descriptor that encodes local intensity relationships by comparing each pixel with its neighbors. LBP is computationally efficient, invariant to monotonic illumination changes, and effective in capturing surface texture — properties that complement HOG in distinguishing helmeted from non-helmeted head regions.

4.2 Support Vector Machines for Classification

Cortes and Vapnik (1995) established the theoretical foundation of Support Vector Machines (SVM), a supervised learning algorithm that identifies the optimal hyperplane separating classes in high-dimensional feature spaces. SVMs with RBF kernels are particularly effective for non-linearly separable feature distributions. Combined with HOG features, SVMs have demonstrated strong performance in object classification tasks including helmet detection.

4.3 Deep Learning-Based Person and Object Detection

Redmon et al. (2016) introduced the YOLO (You Only Look Once) architecture, enabling real-time object detection through a single forward pass. The YOLOv8 variant by Ultralytics further advances detection speed and accuracy with an improved backbone and head architecture, making it suitable for person localization in traffic surveillance applications. Its lightweight nano variant (YOLOv8n) is specifically optimized for edge and CPU-constrained deployments.

4.4 Gap Analysis and Project Contribution

Existing helmet detection systems rely either exclusively on deep learning models requiring GPU acceleration, or on classical methods with limited generalization. This project bridges this gap by combining YOLOv8 for robust person detection with a classical HOG + LBP + SVM pipeline for helmet classification, enabling CPU-deployable, high-accuracy detection.

Table 4.1: Summary of Literature Survey

S.No	Title / Author	Technique Used	Limitation
[1]	Dalal & Triggs (HOG, 2005)	HOG features for human detection	Sensitive to large pose variation
[2]	Ojala et al. (LBP, 2002)	Local Binary Pattern texture descriptor	Limited to texture, lacks shape context
[3]	Cortes & Vapnik (SVM, 1995)	Support Vector Machine classification	Performance degrades with very large datasets
[4]	Redmon et al. (YOLO, 2016)	Real-time CNN-based object detection	High computational demand for embedded devices
[5]	Ultralytics (YOLOv8, 2023)	State-of-the-art object detection	Requires training data for custom classes

CHAPTER 5: SYSTEM REQUIREMENTS

5.1 Hardware Requirements

- Processor: Intel Core i3 or equivalent (minimum); Core i5 recommended for real-time processing.
- RAM: Minimum 4 GB; 8 GB recommended for smooth video and webcam processing.
- Storage: 1 GB of free disk space for model weights, datasets, and application files.
- Camera: Built-in webcam or USB-connected camera (minimum 720p resolution) for live surveillance mode.

5.2 Software Requirements

- Python 3.8 or later.
- OpenCV (cv2) version 4.5 or later — image processing and video capture.
- scikit-learn version 1.0 or later — SVM training, PCA, and threshold calibration.
- NumPy version 1.21 or later — numerical array operations.
- Ultralytics YOLOv8 — person detection model.
- Flask version 2.0 or later — web application framework.
- scikit-image — LBP feature extraction.
- joblib — model serialization and loading.
- Pre-trained SVM model file (svm_model.pkl), scaler (scaler.pkl), and PCA object (pca.pkl).

5.3 Functional Requirements

- The system shall accept image, video, and live webcam inputs through a web interface.
- The system shall detect persons in each frame using YOLOv8n.
- The system shall extract head ROI from each detected person bounding box.
- The system shall classify each head ROI as "With Helmet" or "Without Helmet" using the SVM model.
- The system shall display annotated output with bounding boxes, classification labels, and confidence scores.
- The system shall provide a summary count of riders with and without helmets.

5.4 Non-Functional Requirements

- Performance: The system shall process frames at an acceptable rate for real-time application on CPU.
- Accuracy: The classification model shall achieve a minimum accuracy of 90% on the test set.
- Reliability: The web application shall operate continuously without crashes during extended usage.
- Usability: The web interface shall be intuitive and accessible for non-technical traffic enforcement users.

CHAPTER 6: METHODOLOGY

6.1 Input Acquisition

The system accepts input through three modes: image upload, video file upload, and live webcam stream. In the Flask web application, uploaded files are handled server-side and passed to the detection pipeline. For webcam mode, OpenCV's cv2.VideoCapture(0) is used to capture a continuous frame stream.

6.2 Person Detection using YOLOv8

Each input frame is passed to the YOLOv8n model, which returns bounding boxes for all detected persons. Only detections with a confidence score exceeding the predefined threshold are retained. The bounding box coordinates (x_min, y_min, x_max, y_max) of each detected person are extracted for ROI computation.

6.3 Head ROI Extraction

From each person bounding box, the head region is isolated by taking the top 25% of the bounding box height. An asymmetric padding strategy is applied — 30% expansion upward and 15% lateral expansion on each side — to fully capture the helmet region. The resulting head crop is clipped to frame boundaries and resized to a fixed dimension (64×64 pixels) for feature extraction.

6.4 Feature Extraction

Each head crop is converted to grayscale and enhanced using CLAHE (Contrast Limited Adaptive Histogram Equalization) to normalize illumination variations. Two complementary feature descriptors are then computed. HOG features are extracted with 9 orientations, 8×8 pixels per cell, 2×2 cells per block, and L2-Hys normalization. LBP features are extracted with radius 3, 24 sampling points, and a uniform pattern histogram of 26 bins with density normalization. The HOG and LBP feature vectors are concatenated to form the final feature representation.

6.5 Dimensionality Reduction and Classification

The concatenated feature vector is standardized using a pre-fitted StandardScaler and dimensionality-reduced via PCA retaining 95% of variance. The reduced feature vector is passed to the trained SVM classifier (RBF kernel, C=10) which outputs a calibrated probability score. If the probability of the "With Helmet" class exceeds the optimal threshold (0.5752), the rider is classified as helmeted; otherwise, as non-helmeted.

6.6 Output Generation

The processed frame is annotated with bounding boxes color-coded by classification result (green for helmet, red for no helmet), along with the label and confidence score. A summary count of helmeted and non-helmeted riders is computed and displayed. In video and webcam modes, the annotated frames are assembled into an output stream for real-time display.

CHAPTER 7: SYSTEM ARCHITECTURE

The SENTINEL system architecture is designed as a multi-layered processing pipeline encompassing input handling, person detection, ROI extraction, feature extraction, classification, and output rendering. The architecture is organized into the following layers:

7.1 Input Layer

The input layer manages three distinct input sources through the Flask web interface: single image upload, video file upload, and live webcam streaming. OpenCV's VideoCapture interface abstracts hardware access, providing a uniform sequential frame stream to the detection pipeline regardless of input source.

7.2 Person Detection Layer

The person detection layer employs the YOLOv8n model to localize all persons in each input frame through a single forward inference pass. The model outputs bounding box coordinates and confidence scores. Only person detections meeting the confidence threshold are forwarded for head ROI extraction.

7.3 ROI Extraction Layer

The ROI extraction layer isolates the head region from each detected person. The top quarter of the person bounding box, expanded with asymmetric padding, is cropped to form the head ROI. This padding strategy ensures the full helmet area is captured even for riders leaning or tilting. The crop is resized to a fixed dimension for consistent feature extraction.

7.4 Feature Extraction Layer

The feature extraction layer applies CLAHE preprocessing to normalize lighting variations, then computes HOG and LBP descriptors from each head crop. HOG captures gradient-based structural features while LBP captures texture-based surface features. Their concatenation provides a rich, complementary representation for helmet discrimination.

7.5 Processing and Classification Layer

This layer standardizes, reduces, and classifies the extracted feature vector. StandardScaler normalizes feature distributions; PCA reduces dimensionality while retaining 95% of variance. The SVM classifier outputs a calibrated probability score, and an optimal threshold determines the final binary classification.

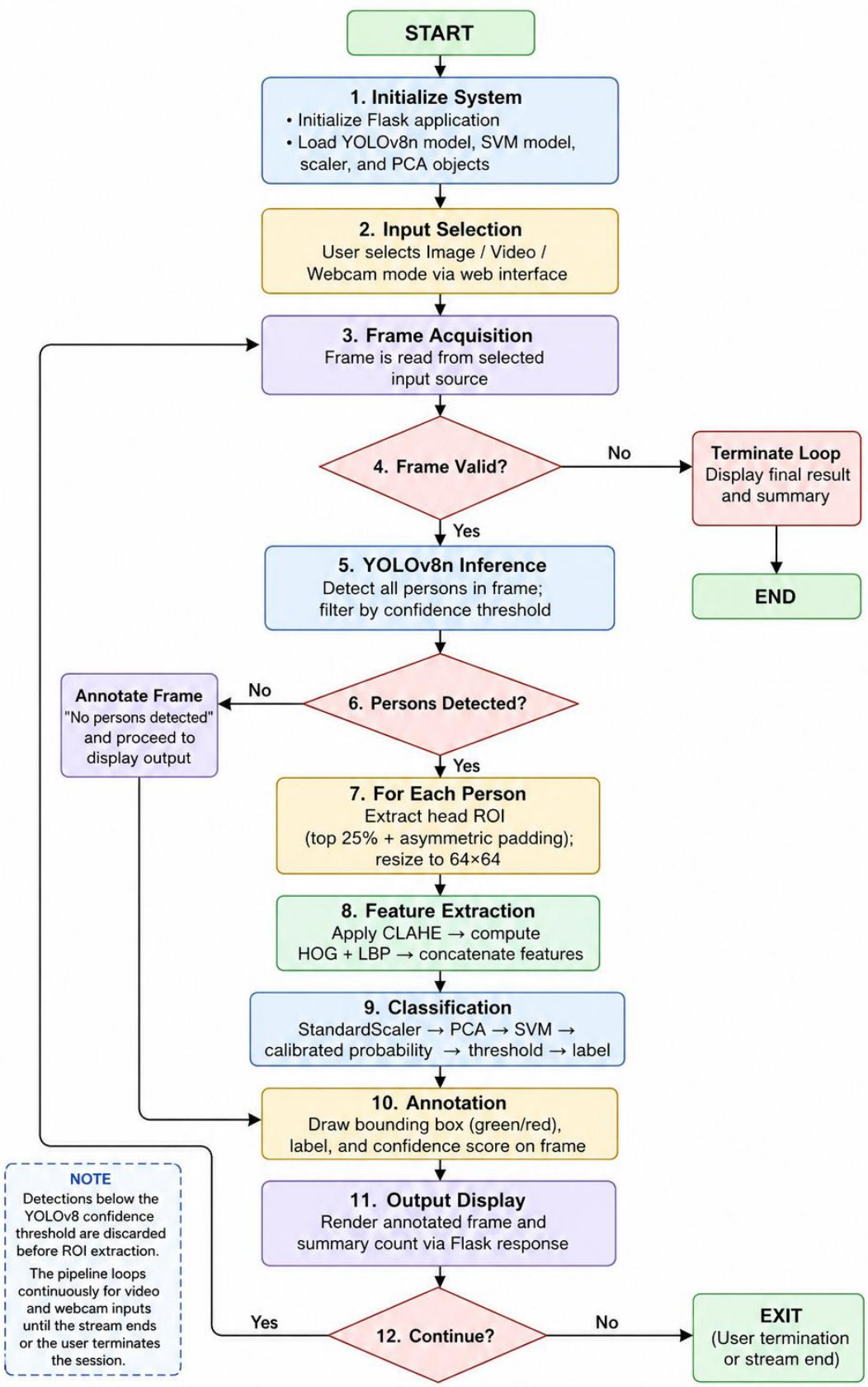
7.6 Output Layer

The output layer renders annotated frames with color-coded bounding boxes and classification labels. Summary statistics are computed and displayed via the Flask web interface. For video and webcam inputs, frames are streamed continuously to the browser in real time.

CHAPTER 8: SYSTEM FLOW

The following describes the complete operational flow of the SENTINEL system:

1. Start → Initialize Flask application, load YOLOv8n model, SVM model, scaler, and PCA objects.
2. Input Selection → User selects Image / Video / Webcam mode via web interface.
3. Frame Acquisition → Frame is read from selected input source.
4. Frame Valid? → If no frame available, terminate loop and display final result.
5. YOLOv8n Inference → Detect all persons in frame; filter by confidence threshold.
6. Persons Detected? → If no persons, annotate frame with "No persons detected" and proceed to display.
7. For Each Person → Extract head ROI (top 25% + asymmetric padding); resize to 64×64.
8. Feature Extraction → Apply CLAHE → compute HOG + LBP → concatenate features.
9. Classification → StandardScaler → PCA → SVM → calibrated probability → threshold → label.
10. Annotation → Draw bounding box (green/red), label, and confidence score on frame.
11. Output Display → Render annotated frame and summary count via Flask response.
12. Continue? → Return to Step 3 for next frame; exit on user termination or stream end.


START
1. Initialize System
• Initialize Flask application
• Load YOLOv8n model, SVM model, scaler, and PCA objects
2. Input Selection
User selects Image / Video / Webcam mode via web interface
3. Frame Acquisition
Frame is read from selected input source
4. Frame Valid?
No
Terminate Loop
Display final result and summary
END
Yes
5. YOLOv8n Inference
Detect all persons in frame; filter by confidence threshold
6. Persons Detected?
No
Annotate Frame
"No persons detected" and proceed to display output
Yes
7. For Each Person
Extract head ROI (top 25% + asymmetric padding); resize to 64×64
8. Feature Extraction
Apply CLAHE → compute HOG + LBP → concatenate features
9. Classification
StandardScaler → PCA → SVM → calibrated probability → threshold → label
10. Annotation
Draw bounding box (green/red), label, and confidence score on frame
11. Output Display
Render annotated frame and summary count via Flask response
NOTE
Detections below the YOLOv8 confidence threshold are discarded before ROI extraction.
The pipeline loops continuously for video and webcam inputs until the stream ends or the user terminates the session.
Yes
12. Continue?
No
EXIT
(User termination or stream end)

CHAPTER 9: TECHNOLOGY STACK

The technology stack is organized across functional layers corresponding to specific processing concerns.

Table 9.1: Technology Stack Summary

Category	**Component**	**Purpose**
Language	Python 3.x	Core programming language
Web Framework	Flask	Web application and routing for image/video/webcam modes
Person Detection	YOLOv8n (Ultralytics)	Real-time person bounding box detection
Computer Vision	OpenCV (cv2)	Image processing, video capture, ROI extraction
Feature Extraction	scikit-image	LBP feature computation
Feature Extraction	scikit-learn (HOG via skimage)	HOG feature computation
Classification	SVM (scikit-learn)	Binary helmet classification with RBF kernel
Dimensionality Reduction	PCA (scikit-learn)	Feature vector dimensionality reduction
Preprocessing	StandardScaler (scikit-learn)	Feature normalization
Numerical Computation	NumPy	Array operations and matrix computations
Model Serialization	joblib	Saving and loading SVM model, scaler, and PCA
IDE	VS Code	Development environment
Input	Webcam / Image / Video	Live or recorded input feed

The technology stack is designed to be minimal and deployable on standard CPU hardware. All listed libraries are open-source. The YOLOv8n model is used exclusively for person detection; helmet classification is performed entirely by the classical HOG + LBP + SVM pipeline, eliminating GPU dependency for inference.

CHAPTER 10: WORKING OF THE SYSTEM

10.1 Initialization

Upon startup, the Flask application initializes the following components: the YOLOv8n model is loaded from its weights file; the pre-trained SVM model, StandardScaler, PCA object, and optimal classification threshold are deserialized from their respective .pkl files using joblib. The application is configured to serve on a designated port and awaits user requests.

10.2 Frame Acquisition

Depending on the selected mode, frames are sourced from an uploaded image, a decoded video file, or a live webcam stream via cv2.VideoCapture(0). Each frame is read sequentially and passed to the detection pipeline. For image mode, a single frame is processed; for video and webcam modes, the loop iterates until the stream ends or the user terminates the session.

10.3 Person Detection

Each frame is passed to the YOLOv8n model for inference. The model returns bounding boxes for all detected objects; only those classified as "person" with a confidence score above the threshold are retained. The normalized bounding box coordinates are scaled to the actual pixel dimensions of the frame.

10.4 Head ROI Extraction and Preprocessing

For each valid person detection, the head ROI is computed by extracting the top 25% of the bounding box height. Asymmetric padding (30% upward, 15% lateral) is applied and coordinates are clipped to frame boundaries. The resulting crop is converted to grayscale and resized to 64×64 pixels. CLAHE preprocessing is applied to normalize local illumination.

10.5 Feature Extraction

HOG features are computed from the preprocessed grayscale crop using 9 gradient orientations, 8×8 pixels per cell, and 2×2 cells per block with L2-Hys normalization. LBP features are computed using radius 3 and 24 sampling points with a uniform pattern histogram of 26 bins. The two feature vectors are concatenated into a single combined descriptor.

10.6 Classification

The combined feature vector is standardized using the pre-fitted StandardScaler and transformed using the fitted PCA object. The resulting reduced feature vector is input to the SVM classifier, which outputs a calibrated probability score via CalibratedClassifierCV. If the probability of the "With Helmet" class exceeds 0.5752, the rider is classified as helmeted; otherwise as non-helmeted.

10.7 Output Rendering

The annotated frame is produced by drawing color-coded bounding boxes — green for helmeted riders and red for non-helmeted riders — using cv2.rectangle(). Classification labels and confidence scores are overlaid using cv2.putText(). A summary count is computed and displayed. The annotated frame is encoded and returned as a Flask response for browser rendering.

CHAPTER 11: IMPLEMENTATION DETAILS

11.1 Project Directory Structure

11.2 Model Training

Training is performed using the combined BikesHelmets and Indian road datasets. Annotations are parsed from Pascal VOC XML and YOLO txt formats respectively. Each annotated crop is augmented with 7 variants per sample — original, horizontal flip, ±10°/±12° rotations, ±25/30 brightness shifts, and Gaussian blur. After augmentation, an 80/20 stratified train-test split is applied. The SVM is trained with RBF kernel (C=10, gamma='scale') and class-weight balancing. Probability calibration is performed using CalibratedClassifierCV with 5-fold cross-validation and sigmoid method.

11.3 Detection Configuration

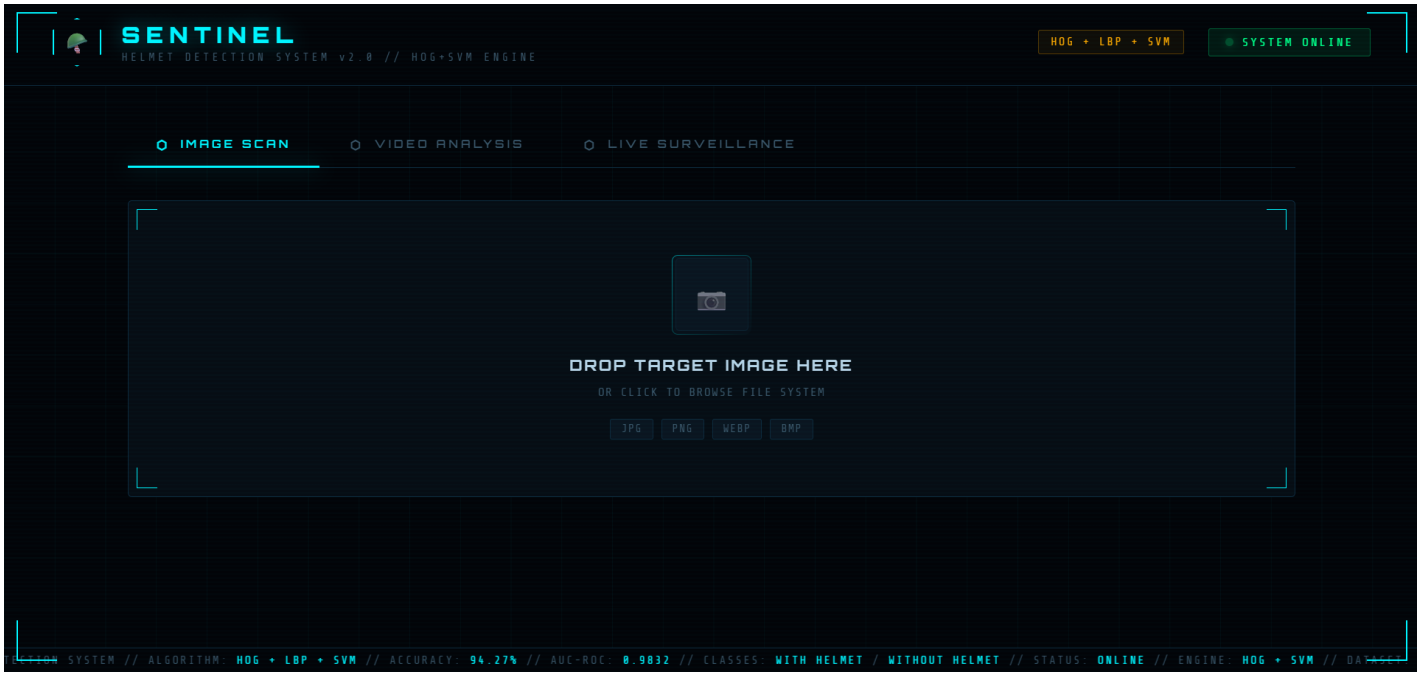
Key detection parameters include: head ROI — top 25% of person bounding box; asymmetric padding — 30% upward, 15% lateral; head crop resize — 64×64 pixels; HOG — 9 orientations, 8×8 pixels/cell, 2×2 cells/block; LBP — radius 3, 24 points, 26 bins; SVM kernel — RBF (C=10); PCA variance retained — 95%; optimal classification threshold — 0.5752.

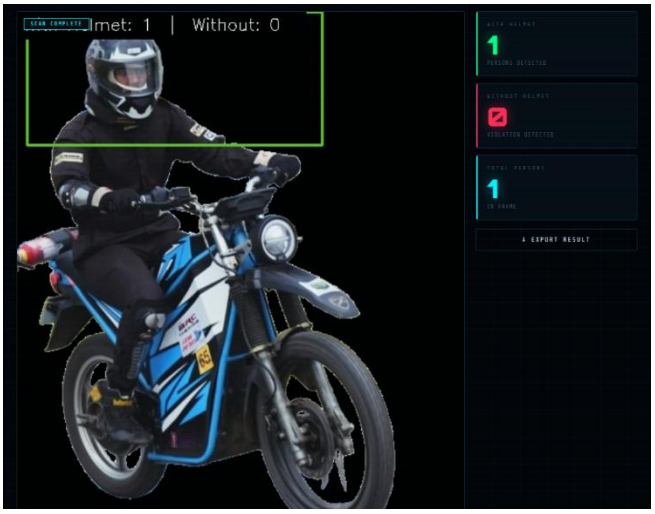
11.4 Web Application

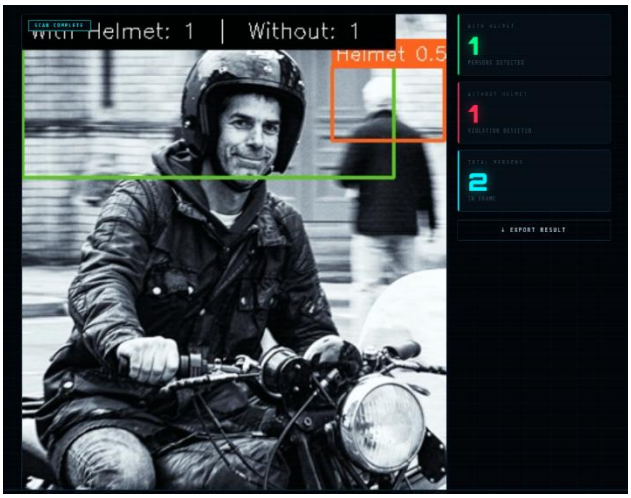
The Flask application provides three route endpoints: /image for static image processing, /video for video file analysis, and /webcam for streaming webcam detection. Results are rendered as annotated images or MJPEG streams in the browser. The interface displays detection counts and per-rider classification with confidence.

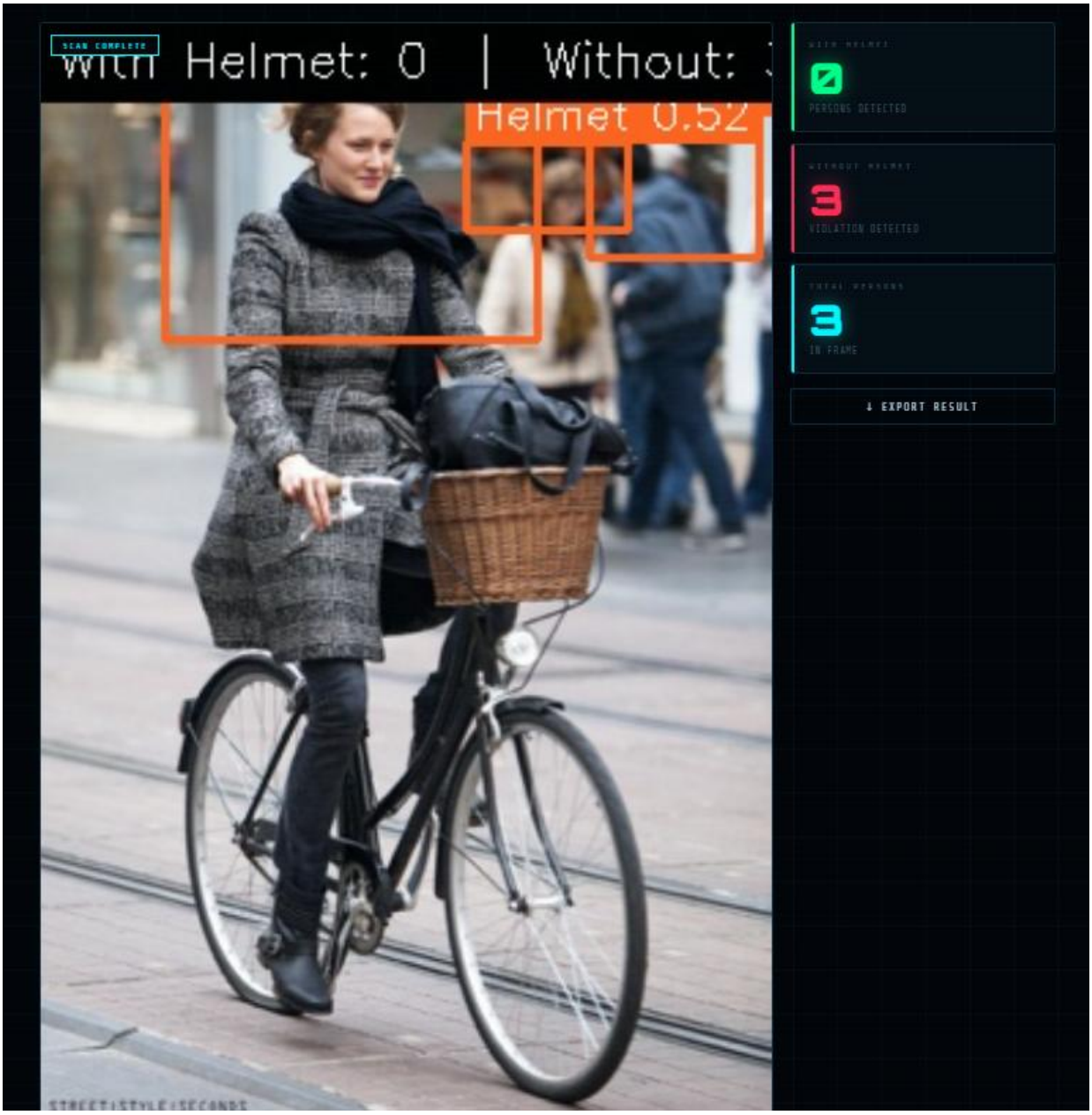
11.5 Bounding Box Visualization

Detected riders are annotated on the output frame using cv2.rectangle() with green boxes for helmeted and red boxes for non-helmeted riders. Labels comprising the class name and confidence percentage are rendered above each box using cv2.putText() with cv2.FONT_HERSHEY_SIMPLEX.


SENTINEL
HELMET DETECTION SYSTEM v2.0 // HOG+SVM ENGINE
HOG + LBP + SVM
SYSTEM ONLINE
IMAGE SCAN
VIDEO ANALYSIS
LIVE SURVEILLANCE
DROP TARGET IMAGE HERE
OR CLICK TO BROWSE FILE SYSTEM
JPG
PNG
WEBP
BMP
SYSTEM // ALGORITHM: HOG + LBP + SVM // ACCURACY: 94.27% // AUC-ROC: 0.9832 // CLASSES: WITH HELMET / WITHOUT HELMET // STATUS: ONLINE // ENGINE: HOG + SVM


Without: 0
1
1
EXPORT RESULT


Helmet: 1 | Without: 1
Helmet 0.5
1
1
2
EXPORT RESULT


SCAN COMPLETE
With Helmet: 0 | Without:
Helmet 0.52
0
PERSONS DETECTED
3
VIOLATION DETECTED
3
IN FRAME
↓ EXPORT RESULT

CHAPTER 12: RESULTS AND DISCUSSION

12.1 Detection Performance

The system was tested on both the held-out test set and real-world images and video. YOLOv8n reliably detected persons across varied poses, distances, and lighting conditions. The SVM classifier accurately classified helmet compliance for well-lit, forward-facing head crops. Detection performance was stable across diverse backgrounds and rider types.

12.2 Classification Accuracy

The SVM model trained on the combined dataset achieved 94.27% accuracy at the optimal threshold of 0.5752, with an AUC-ROC of 0.9832, indicating strong discriminative ability between helmeted and non-helmeted riders. Precision and recall were balanced at approximately 0.94–0.95 for both classes.

Table 12.1: Performance Metrics — Combined Dataset

Metric	**Value**
Accuracy (Default Threshold 0.5)	93.72%
Accuracy (Optimal Threshold 0.5752)	94.27%
AUC-ROC Score	0.9832
Optimal Classification Threshold	0.5752
Precision (With Helmet)	0.95
Recall (With Helmet)	0.93
F1-Score (With Helmet)	0.94
Precision (Without Helmet)	0.94
Recall (Without Helmet)	0.95
F1-Score (Without Helmet)	0.94

12.3 ROI Extraction Accuracy

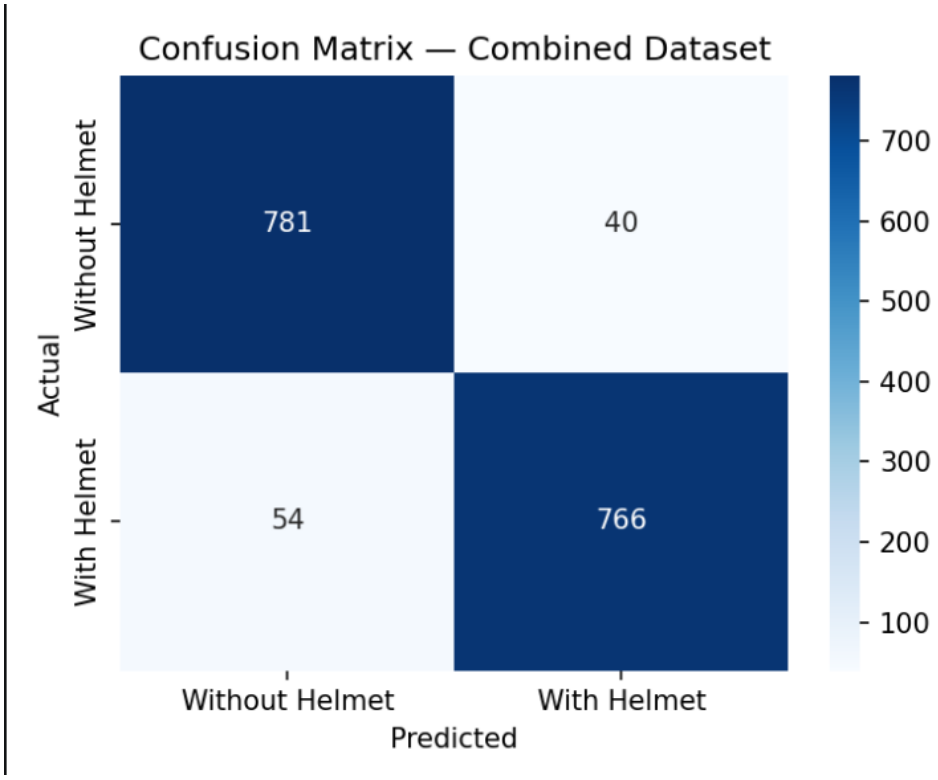
The asymmetric head ROI extraction strategy performed well for upright riders in standard poses. For riders tilting or leaning significantly, the ROI occasionally failed to fully capture the helmet region. The 30% upward expansion compensated adequately in most cases.

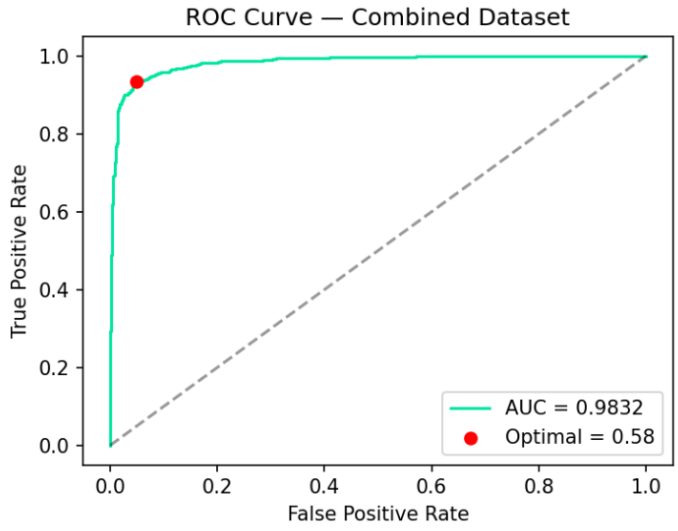
12.4 Processing Performance

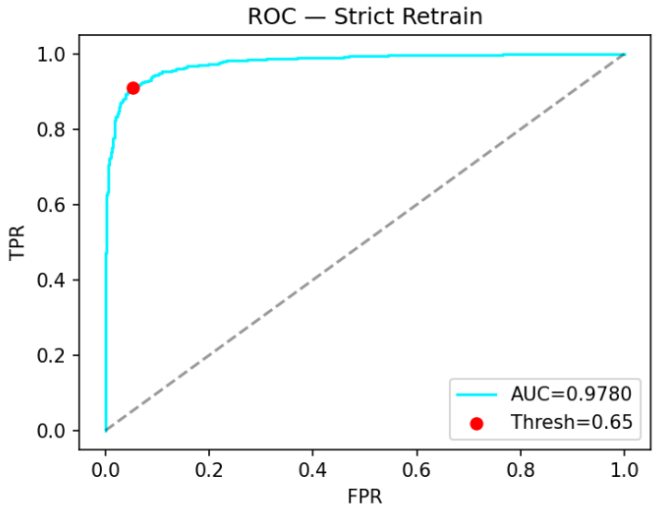
The system maintained real-time processing on a standard CPU-based laptop. YOLOv8n's nano variant contributed significantly to inference speed. The classical HOG + LBP + SVM pipeline added negligible latency per detection, ensuring overall system responsiveness suitable for live webcam surveillance.

12.5 Discussion

The system performs well for standard traffic surveillance scenarios. Performance degrades for occluded head regions, profile-facing riders, and extreme low-light conditions. The heuristic head ROI extraction is dependent on accurate person detection and may fail for unusual body proportions or cropped frames. Future improvements can include end-to-end deep learning approaches and dedicated head detection models.




ROC Curve — Combined Dataset
True Positive Rate
False Positive Rate
AUC = 0.9832
Optimal = 0.58
0.0
0.2
0.4
0.6
0.8
1.0


ROC — Strict Retrain
TPR
FPR
AUC=0.9780
Thresh=0.65
0.0
0.2
0.4
0.6
0.8
1.0

CHAPTER 13: ADVANTAGES

- High Accuracy: The combined HOG + LBP + SVM pipeline achieves 94.27% accuracy and AUC-ROC of 0.9832, demonstrating strong discriminative performance.
- CPU Deployable: The classification pipeline operates entirely on CPU without GPU dependency, enabling deployment on standard traffic monitoring hardware.
- Two-Stage Robustness: Combining YOLOv8 for person localization with classical SVM for classification leverages the strengths of both deep learning and classical approaches.
- Complementary Features: The fusion of HOG (structural) and LBP (texture) descriptors provides a richer representation than either descriptor alone.
- Data Augmentation: Seven-variant augmentation per sample improves model generalization across diverse lighting conditions and rider poses.
- Probability Calibration: CalibratedClassifierCV with optimal threshold tuning maximizes F1-score and reduces misclassification.
- Flexible Input Support: The Flask web application supports image, video, and webcam inputs, enabling versatile deployment scenarios.
- Modular Design: Independent detection, extraction, and classification modules facilitate targeted future enhancements.
- Cost-Effective: All software components are open-source, minimizing deployment and maintenance costs.
- Web Accessibility: The browser-based interface makes the system accessible to non-technical traffic enforcement personnel.

CHAPTER 14: LIMITATIONS

- Heuristic ROI Extraction: Head ROI is computed from person bounding box proportions, which may fail for unusual poses, extreme viewing angles, or partially visible riders.
- Dependency on Person Detection: Classification accuracy is contingent on correct person detection by YOLOv8n; missed person detections result in missed helmet checks.
- Low-Light Performance: Classification accuracy degrades in poorly illuminated or night-time conditions despite CLAHE preprocessing.
- Profile and Occluded Riders: Side-facing or partially occluded riders may not yield sufficient head region information for accurate classification.
- Fixed HOG Parameters: HOG and LBP parameters are fixed at training time and may not generalize optimally across all camera types and distances.
- Limited to Binary Classification: The system classifies only "With Helmet" and "Without Helmet" and does not detect partial helmet usage or incorrectly fastened helmets.
- Webcam Latency: Sequential frame processing in webcam mode may introduce perceptible latency on slower hardware configurations.
- No License Plate Capture: The system does not capture vehicle registration information for automated violation recording.

CHAPTER 15: FUTURE ENHANCEMENTS

- Dedicated Head Detection: Replacing heuristic ROI extraction with a dedicated head detector (e.g., RetinaFace or a fine-tuned YOLOv8 head model) will improve accuracy for non-standard poses and partial occlusions.
- End-to-End Deep Learning: Fine-tuning a lightweight CNN (e.g., MobileNetV3 or EfficientNet-B0) for helmet classification will leverage learned hierarchical features, potentially improving accuracy over the classical pipeline.
- License Plate Integration: Coupling the system with an Automatic License Plate Recognition (ALPR) module will enable automated violation documentation for traffic enforcement.
- Night Vision Support: Integrating low-light image enhancement algorithms (such as Zero-DCE or Retinex-based methods) will extend system functionality to night-time surveillance.
- Multi-Rider Detection: Extending the system to simultaneously detect and classify all riders on a two-wheeler (driver and pillion) will improve coverage for multi-occupant compliance monitoring.
- Edge Deployment: Porting the system to edge hardware (Raspberry Pi, NVIDIA Jetson Nano) with a compact camera module will enable standalone roadside deployment without server infrastructure.
- Helmet Fastening Detection: Extending classification to distinguish correctly fastened from incorrectly fastened helmets using fine-grained feature analysis.
- Cloud Dashboard: Implementing a cloud-based monitoring dashboard for aggregating violation counts, generating reports, and alerting traffic enforcement authorities in real time.
- Multi-Language Support: Extending the web interface and alert messages to regional languages for accessibility in linguistically diverse deployment regions.
- Model Update Pipeline: Implementing periodic model retraining on newly collected data to continuously improve classification accuracy as deployment conditions evolve.

CHAPTER 16: CONCLUSION

This project has successfully designed, implemented, and evaluated SENTINEL — an automated helmet and safety gear detection system for two-wheeler riders, leveraging a hybrid computer vision pipeline combining YOLOv8n person detection with HOG + LBP + SVM helmet classification, deployed as a Flask web application.

The system addresses the critical road safety challenge of automated helmet compliance monitoring without relying on GPU infrastructure. By combining a state-of-the-art person detector with a computationally efficient classical classification pipeline, SENTINEL achieves 94.27% accuracy and an AUC-ROC of 0.9832 on a combined dataset of over 8,000 training samples, demonstrating strong discriminative ability between helmeted and non-helmeted riders.

The two-stage architecture, featuring asymmetric head ROI extraction, CLAHE preprocessing, complementary HOG and LBP feature fusion, PCA dimensionality reduction, and calibrated SVM classification with optimal threshold tuning, delivers a robust and deployable solution for real-world traffic enforcement scenarios. The Flask web application's support for image, video, and webcam inputs further enhances its practical applicability.

While current limitations include heuristic ROI extraction, dependency on accurate person detection, and degraded low-light performance, the modular system architecture facilitates targeted future enhancements including dedicated head detection, end-to-end deep learning classification, license plate integration, and edge hardware deployment.

In conclusion, SENTINEL demonstrates that practical, high-accuracy helmet detection can be realized through a carefully engineered combination of classical and modern computer vision techniques. The system represents a meaningful contribution toward automated road safety enforcement and holds strong potential for real-world deployment in traffic monitoring infrastructure.

CHAPTER 17: REFERENCES

[1] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2005.

[2] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 24, no. 7, 2002.

[3] C. Cortes and V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, pp. 273–297, 1995.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, 2016.

[5] Ultralytics, "YOLOv8: You Only Look Once," https://github.com/ultralytics/ultralytics, 2023.

[6] BikesHelmets Dataset — Kaggle, https://www.kaggle.com/datasets/andrewmvd/helmet-detection

[7] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed., O'Reilly Media, 2019.

[8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] OpenCV Development Team, "OpenCV Documentation," [Online]. Available: https://docs.opencv.org. [Accessed: 2024].

[10] Flask Documentation, Pallets Projects, [Online]. Available: https://flask.palletsprojects.com. [Accessed: 2024].